応用編７日目
まとめ

一週間で身につくJava言語

例外処理

- SampleEx701.java参照
- 例外とはプログラム動作中に起こる異常。
- 実行時エラーともいう。
- 理由は様々
  - 0による割り算が行われたため
  - 配列の範囲から外れてアクセスした

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at exday7.SampleEx701.main(SampleEx701.java:8)
```

0 で割り算を行った時の例外

例外と例外処理

- 例外が発生したの処理　…　**例外処理（れいがいしょり）**
- プログラム中に対処方法を記述できる

処理方法	内容
try～catch	例外を、tryの{}内で起こった例外を、catch以下で処理します
throws	発生した例外を、他のクラスにゆだねます

try/catchによる例外処理①

- try～catchは、{}内で例外が発生した場合、それに対処する処理を記述するためのもの

try～catchの書式

```
try{
    (処理①)
}catch((例外クラス) 変数){
    (処理②)
}finally{
    (処理③)
}
```

- SampleEx702参照

try/catchによる例外処理②

```
package exday7;

public class SampleEx702 {

    public static void main(String[] args) {
        try{
            for(int i = 3; i >= 0; i--){
                int a = i;
                int b = 5;
                System.out.print(b + " / " + a + " = ");
                System.out.println(b / a);
            }
        }catch(ArithmeticException e){
            System.out.println();
            System.out.println("0による割り算発生");
        }finally{
            System.out.println("終了");
        }
    }

}
```

例外発生

throws

- **throws**キーワードは、メソッド内で発生した例外を、外部にその処理をゆだねるためのもの

throwsキーワードの書式

```
メソッド名　(引数型　引数名)　throws　例外クラス型, …
```

- SampleEx703参照

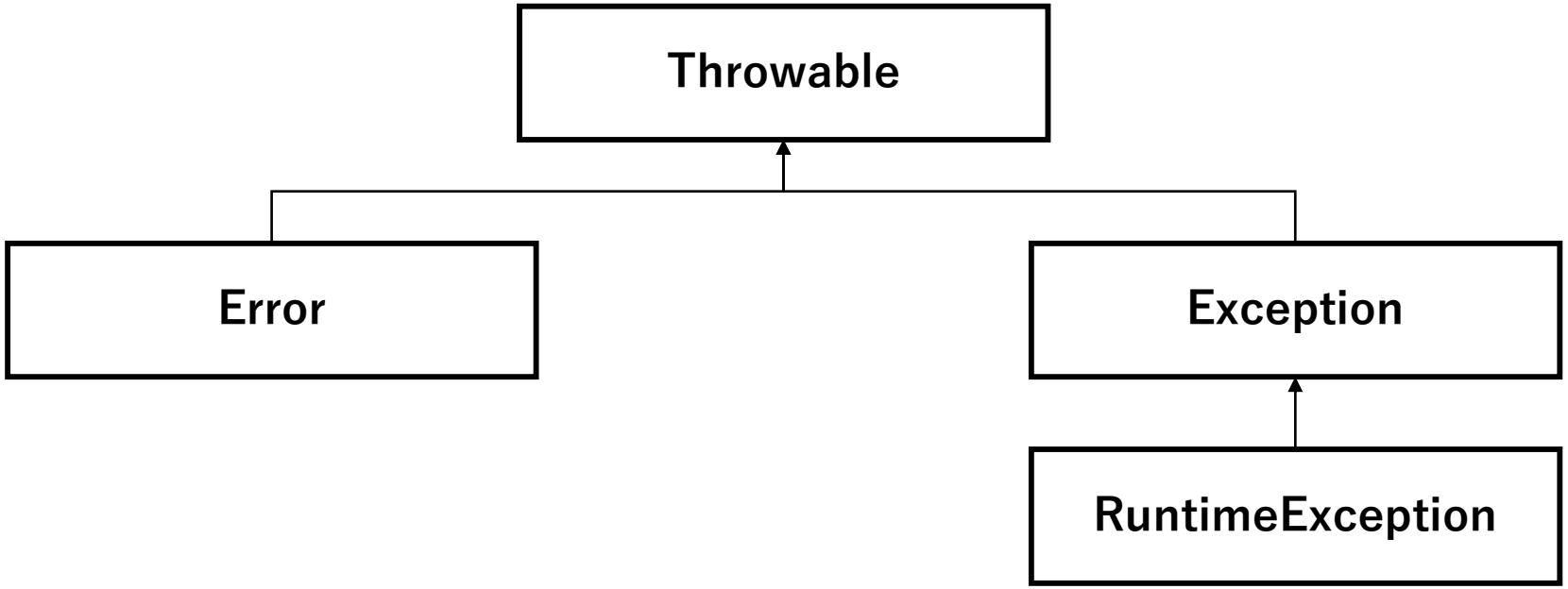
throwsによる例外処理

```
package exday7;

public class SampleEx703 {

    public static void main(String[] args) {
        try{
            for(int i = 0; i<= 5; i++){
                int a = getNum(i);
                int b = 5;
                System.out.print(a + " / " + b + " = ");
                System.out.println(calc(a,b));
            }
        }catch(ArithmeticException e){
            System.out.println();
            System.out.println("0による割り算発生");
        }catch(ArrayIndexOutOfBoundsException e){
            System.out.println("配列の範囲外にアクセスしました");
        }finally{
            System.out.println("終了");
        }
    }
    //
    private static int calc(int a,int b) throws ArithmeticException{
        return a / b;
    }
    //  範囲外に出たときの処理
    public static int getNum(int index) throws ArrayIndexOutOfBoundsException{
        int[] num = { 1, 2, 3, 4 };
        return num[index];
    }

}
```

例外発生

例外関連クラスの種類

- 例外クラスは**Exception**クラスを継承している
- Exception、Errorクラスは、**Throwable**を継承
- RuntimeException はExceptionを継承したもの



Javaの例外処理に関連するクラス

クラス名	働き	主なサブクラス
Throwable	全てのエラーおよび例外のスーパークラス 通常、プログラム内で直接使用しない	
Error	本来アプリケーションで発生してはならない重大なエラーを表現 プログラム内で拡張したり、サブクラスを定義することはない	NoClassDefFoundError OutOfMemoryError
Exception	一般的な例外を扱うクラス Javaで発生する多くの例外はこのクラスを継承して使用されます。このクラスを継承し、独自のクラスを作成することがあります。 このクラスを直接継承している例外のことを、**検査例外（チェック例外）**と言う。	ClassNotFoundException CloneNotSupportedException DataFormatException FileNotFoundException
RuntimeException	Exceptionのサブクラス 一般的にビルド時にエラーとして検出しきれないプログラムのミスにより実行時に発生する例外クラス**非検査例外**と言う。	ClassCastException IllegalArgumentException IndexOutOfBoundsException NullPointerException

検査例外と非検査例外

- **Exception**を直接継承した例外クラス　…　**検査例外**
- **RuntimeException**を継承した例外クラス　…　**非検査例外**
  - ArrayIndexOutofBoundException
  - など
- 検査例外が発生する可能性がある場合**必ず例外処理を記述する**
- SampleEx704参照

throw

- ユーザーが独自に作成した例外を発生させるには、**throw**を用いる
- **throw**は、ユーザー定義されているものに限らず、例外を発生させることができる

throwで例外を発生させる

```
throw new 例外クラス名(コンストラクタの引数,…);
```